

## Week 3 - Class 5: Introduction to Pointers and References

### Why This Matters

Up until now, variables have directly stored values. But sometimes we need to store memory addresses or create aliases. C++ provides two tools: pointers (store addresses) and references (aliases).

### The Symbols \* and & in C++

- & in expressions: address-of operator (&x means address of x).
- & in declarations: declares a reference (int& ref = x).
- \* in expressions: dereference operator (\*p gives value pointed to).
- \* in declarations: declares a pointer (int\* p).

### What Is a Pointer?

A pointer is a variable that stores the address of another variable.

Example:

```
int x = 10;
int* p = &x;
```

- &x is address of x
- p stores that address
- \*p accesses the value at the address

### ASCII Memory Diagram (Pointer)

```
x: [10] (int value)
p: [&x] -> address of x
*p: 10 (value through pointer)
```

### Example: Pointer Basics

```
#include <iostream>
using namespace std;

int main() {
    int x = 42;
    int* p = &x;

    cout << "x = " << x << "\n";
    cout << "p = " << p << "\n";
    cout << "*p = " << *p << "\n";

    *p = 99;
    cout << "x (after *p=99) = " << x << "\n";
}
```

### Null Pointers

A pointer can represent 'no address'. Use nullptr.

Example: int\* p = nullptr;

### What Is a Reference?

A reference is an alias: another name for an existing variable.

Example:

```
int y = 5;
int& ref = y;
```

## C++14 Fundamentals - Student Edition

ref is just another name for y. Changing ref changes y.

### References and Memory Usage

A reference does NOT create a new copy of the variable. It is simply another label for the same memory location.

Example:

```
int a = 10;
int& r = a;
```

Memory:

```
+-----+
| a: 10 |
+-----+
```

ref: alias -> same box as a

Behind the scenes, compilers may implement a reference like a constant pointer, but your program does not pay extra memory cost.

Whether you access a or r, only one int exists in memory.

### Example: References

```
#include <iostream>
using namespace std;

int main() {
    int y = 10;
    int& ref = y;

    cout << "y = " << y << "\n";
    cout << "ref = " << ref << "\n";

    ref = 20;
    cout << "y (after ref=20) = " << y << "\n";
}
```

### ASCII Diagram: Reference vs Pointer

Reference:

y: [10]

ref: alias -> same box as y

Pointer:

y: [10]

p: [&y] -> separate box storing address

### When to Use Pointers vs References

- Use reference: when always bound to a valid object, simpler syntax, no reseating needed.
- Use pointer: when reseating or null possible, when working with C APIs, or for manual memory management.

### Key Differences

- Reference must be initialized, cannot resear, cannot be null.
- Pointer can resear, can be null, requires \* to access value.

### Common Mistakes

1. Forgetting \* when dereferencing pointers.

## C++14 Fundamentals - Student Edition

2. Confusing pointer vs pointed value.
3. Assuming reference is a copy (it's just an alias).
4. Leaving pointer uninitialized (dangerous).

### Exercises

1. Create int x and pointer p. Print x, p, \*p.
2. Modify x via \*p.
3. Create int y and reference ref. Print and modify.
4. Draw memory diagrams comparing pointer and reference.
5. Write a short paragraph: when would you choose pointer vs reference?

### Project Tie-In

Pointer Playground Part 1: print addresses, dereference pointers, use references as aliases.